The Anatomy of a Production-Grade FSM

- The **Event**: An occurrence that requests a change in system behavior.
- The **State**: Defines the current **behavior of the system**.

It determines which events are meaningful and how they should be handled.

🧠 The Shift in Mindset:

A real **FSM** fundamentally asks a different question:

"What am I **doing now**, and does this **event matter** in my current situation?"

The 4-Question Pre-Code Checklist

Before writing code, **list your states**. For each state, ask:

1. What events are **accepted**?
2. What events are **ignored**?
3. What action **happens**?
4. What is the next state?

🚀 I found out this is used to build mission-critical systems in: Automotive ECUs · Industrial Controllers · Medical Devices · Aerospace Systems

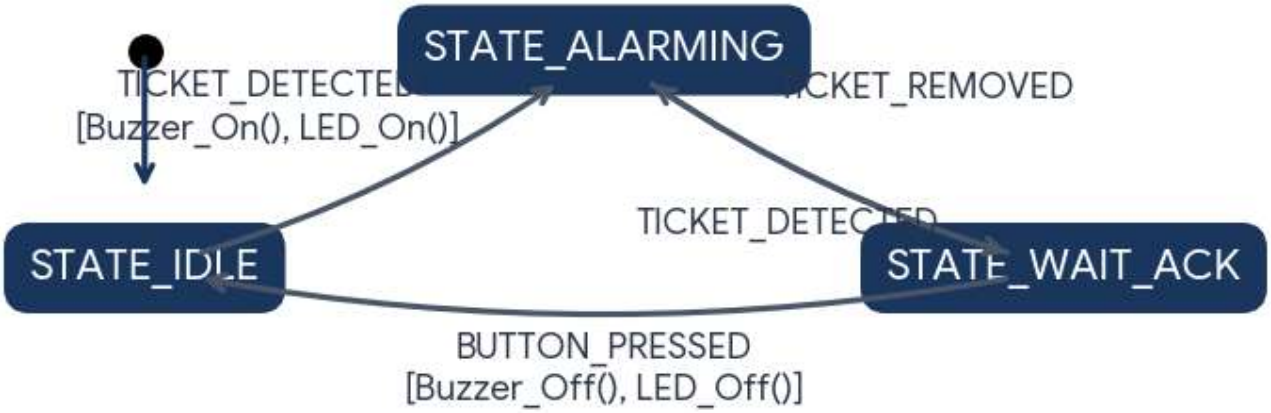
```c
void FSM_ProcessEvent(SystemEvent_t event) {
    switch (current_state) {
        case STATE_IDLE:
            if (event == TICKET_DETECTED) {
                Buzzer_On(); LED_On();
                current_state = STATE_ALARMING;
            }
            break;

        case STATE_ALARMING:
            if (event == TICKET_REMOVED) {
                current_state = STATE_WAIT_ACK;
            }
            break;

        case STATE_WAIT_ACK:
            if (event == BUTTON_PRESSED) {
                Buzzer_Off(); LED_Off();
                current_state = STATE_IDLE;
            } else if (event == TICKET_DETECTED) {
                current_state = STATE_ALARMING;
            }
            break;

        default:
            Buzzer_Off(); LED_Off();
            current_state = STATE_IDLE;
            break;
    }
}

```


STATE_ALARMING
TICKET_DETECTED
[Buzzer_On(), LED_On()]
TICKET_REMOVED
TICKET_DETECTED
STATE_IDLE
STATE_WAIT_ACK
BUTTON_PRESSED
[Buzzer_Off(), LED_Off()]